

Practica SD

Índice

1. Introducción del Escenario de Pruebas	3
1.1. Contexto del Proyecto y Objetivos	3
1.2. Ficha Técnica del Entorno Físico (Hardware y Red)	4
1.3. Entorno e Infraestructura de Red	4
1.4. Resultados Obtenidos	5
2. Análisis del Modelo Síncrono/Directo (Pyro4)	6
2.1. Arquitectura de Comunicación y Acoplamiento Síncrono	6
2.2. La Barrera del RTT (Round Trip Time)	6
2.3. Ineficacia de la Escalabilidad Horizontal en Entornos Síncronos	7
3. Análisis del Modelo Asíncrono / Indirecto (RabbitMQ + Redis)	8
3.1. Filosofía de Desacoplamiento Temporal y Espacial	8
3.2. Paralelismo Real y Escalabilidad Horizontal de los Workers	8
3.3. Resultados No Bloqueante en Redis	9
4. El Impacto de la Contención de Datos (Comparativa 20k vs. 60k)	10
4.1. Análisis del Escenario Sin Numerar (20k Tickets): Escalabilidad Limpia	10
4.2. Análisis del Escenario Numerado (60k Tickets): El Límite de la Saturación	10
5. Conclusiones Arquitectónicas Finales	12
5.1. Cuadro Comparativo de Rendimiento y Escalabilidad	12
5.2. Superioridad del Desacoplamiento Temporal	12
5.3. El Rol Crítico de la Base de Datos	12

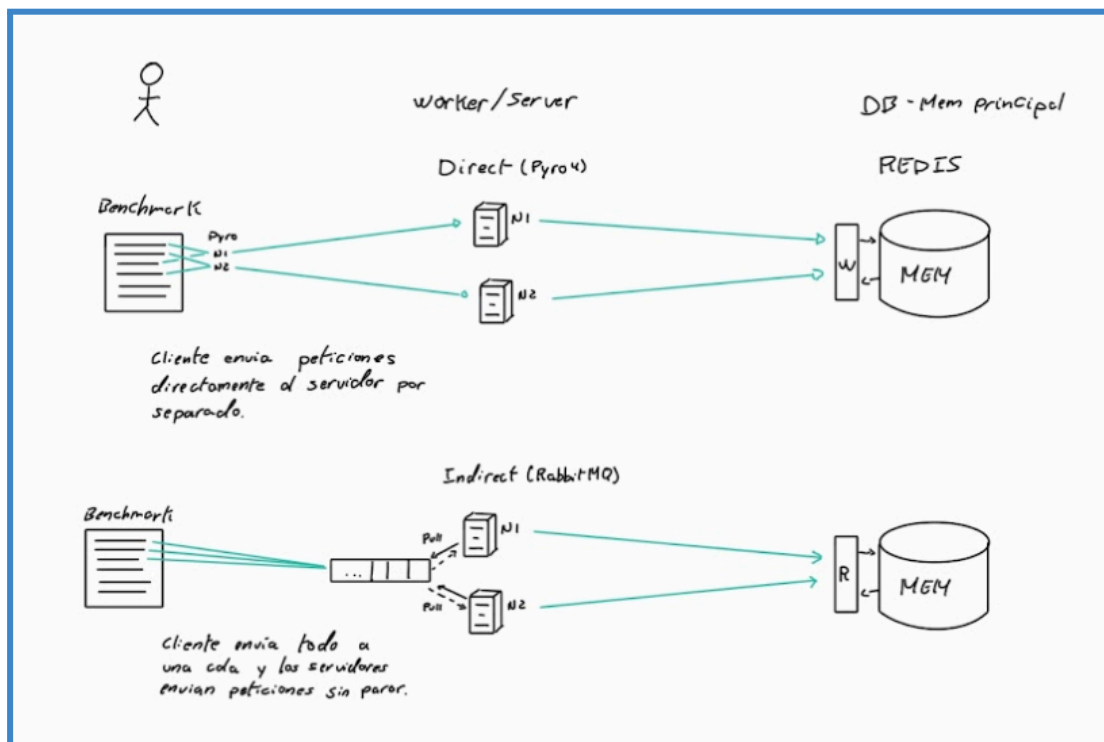
1. Introducción del Escenario de Pruebas

1.1. Contexto del Proyecto y Objetivos

El informe documenta el análisis y la evaluación de rendimiento de un sistema distribuido de reserva y compra masiva de entradas para eventos. El problema principal de esta práctica radica en diseñar una arquitectura capaz de mitigar las problemáticas clásicas de los sistemas concurrentes distribuidos, la alta disponibilidad, la tolerancia a fallos, la consistencia de los datos frente a condiciones de carrera (por ejemplo, la sobreventa de un mismo asiento) y, fundamentalmente, la **escalabilidad horizontal**.

Para abordar este reto, se han implementado y evaluado dos modelos arquitectónicos con filosofías de comunicación opuestas:

1. **Modelo Síncrono / Directo:** Basado en llamadas a procedimientos remotos (RPC) mediante la librería Pyro4, donde el cliente interactúa de forma acoplada y directa con un conjunto de nodos servidores balanceados mediante una estrategia de *Round-Robin* hacia una base de datos en memoria Redis.
2. **Modelo Asíncrono / Indirecto:** Basado en una arquitectura orientada a mensajes, utilizando RabbitMQ como intermediario de mensajería (*Message Broker*) encargado de encolar las peticiones y distribuir las a un conjunto de procesos trabajadores (*Workers*), los cuales persisten el estado del inventario de forma atómica en una base de datos en memoria Redis.



El objetivo es inyectar cargas de trabajo masivas de manera concurrente (baterías de test de 20.000 y 60.000 operaciones) variando de forma controlada el número de procesos asignados en el servidor (1, 2, 4, 6 y 8 instancias) para extraer conclusiones sólidas sobre el límite de rendimiento (*throughput*) y la capacidad de escalabilidad de cada paradigma.

1.2. Ficha Técnica del Entorno Físico (Hardware y Red)

Para simular un entorno distribuido real y evitar los sesgos de contención de recursos que provocaría ejecutar toda la infraestructura en un único computador (*localhost*), las pruebas se han desplegado de forma nativa cruzando dos máquinas físicas independientes conectadas en red local (LAN).

Máquina Servidor (Torre Backend):

Esta máquina actúa como núcleo del sistema, soportando el almacenamiento de datos, las colas de mensajes, el servidor de nombres de Pyro4 y el procesamiento backend.

- **Procesador (CPU):** Intel i5 13600 KF.
- **Memoria RAM:** 16 GB DDR4 a 3200 MHz funcionando en configuración *Dual Channel*.
- **Almacenamiento:** Unidad de estado sólido (SSD) M.2 NVMe PCIe Gen3
- **Sistema Operativo:** Windows 11 Pro.
- **Infraestructura de Software (Contenedores):**
 - Motor de Docker levantando una instancia oficial de **Redis** (Puerto 6379).
 - Contenedor Docker oficial de **RabbitMQ** versión 3-management (Puerto de datos 5672, puerto de gestión 15672).

Máquina Cliente (Portátil Inyector de Carga):

Esta máquina se encarga de emular la concurrencia de miles de usuarios leyendo los archivos de benchmark y lanzando las ráfagas de peticiones de compra de manera ininterrumpida.

- **Procesador (CPU):** Intel Core i7-1355U.
- **Memoria RAM:** 16 GB DDR4 a 3200 MHz.
- **Almacenamiento:** Unidad de estado sólido Sata (SSD).
- **Sistema Operativo:** Windows 11 Home.

1.3. Entorno e Infraestructura de Red

La infraestructura que comunica la Máquina Cliente (Portátil) con la Máquina Servidor (Torre, configurada bajo la dirección IP local 192.168.1.140) se ha establecido bajo los siguientes parámetros de conectividad:

- **Topología y Medio Físico:** Conexión híbrida a través de un router doméstico. La Máquina Servidor está conectada mediante cable Ethernet (1000 Mbps), garantizando una tasa de transferencia estable en el backend. La Máquina Cliente está enlazada mediante conectividad inalámbrica Wi-Fi 6.
- **Métricas de Red Estimadas:**
 - Ancho de banda efectivo de red local superior a los 670 Mbps concurrentes.
 - Latencia base de red (*Ping*) constante con valores medios entre 1 ms y 3.5 ms.

Esta latencia base de red local, aunque aparentemente insignificante en entornos ofimáticos, constituirá el pilar fundamental para justificar el comportamiento de la arquitectura síncrona frente a la asíncrona a medida que avancemos en la documentación.

1.4 Resultados Obtenidos

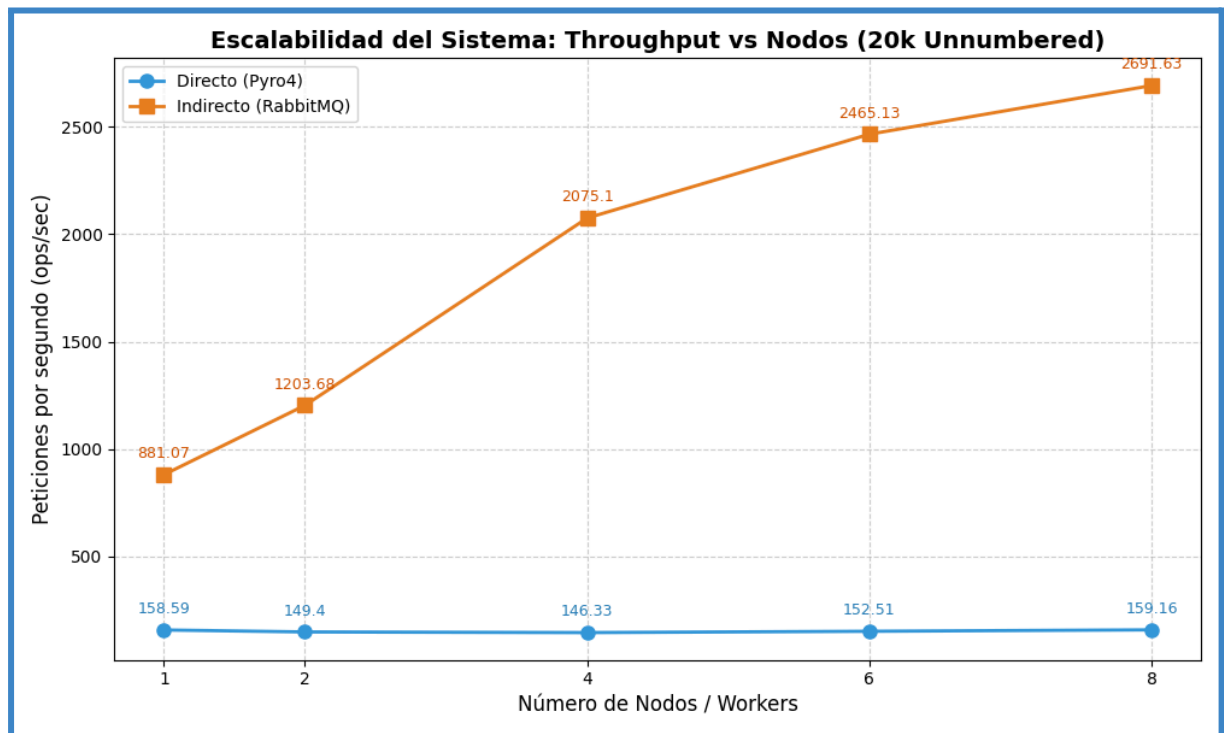


Figura 1: Evolución del Throughput (ops/sec) en función del número de nodos concurrentes para una carga de 20.000 peticiones sin numerar.

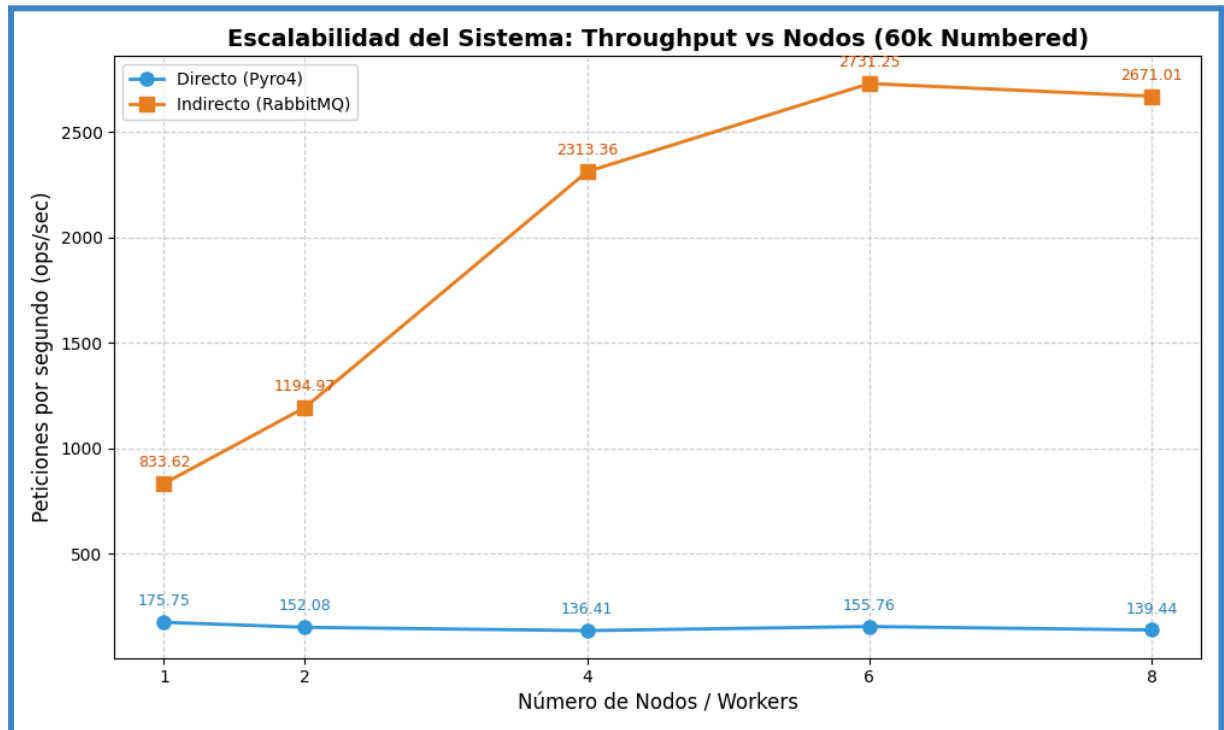


Figura 2: Evolución del Throughput (ops/sec) en función del número de nodos concurrentes para una carga de 60.000 asientos numerados.

2. Análisis del Modelo Síncrono/Directo (Pyro4)

2.1. Arquitectura de Comunicación y Acoplamiento Síncrono

La primera solución evaluada se fundamenta en un modelo de comunicación directa mediante llamadas a procedimientos remotos (RPC), implementado a través de la librería Pyro4. En esta arquitectura, los objetos que contienen la lógica de negocio (TicketWorker) se registran en un Servidor de Nombres centralizado, y la Máquina Cliente (Portátil) interactúa directamente con ellos obteniendo proxies remotos que simulan la presencia local de las funciones de compra (buy_unnumbered y buy_numbered). El núcleo del rendimiento de este modelo está determinado por su naturaleza síncrona y fuertemente acoplada. El script inyector de carga (client_direct_benchmark.py) procesa de manera secuencial y en un único hilo de ejecución cada línea del archivo de transacciones. El flujo temporal de cada petición responde a un ciclo estrictamente bloqueante:

1. El cliente lee la transacción del benchmark.
2. El cliente invoca el método remoto a través del proxy de Pyro4.
3. Bloqueo del hilo: La ejecución del cliente se congela por completo. El hilo cede el control y espera de forma pasiva a que los datos viajen por la red, sean procesados por el nodo de la Torre, se verifiquen los datos en Redis, se genere la respuesta y esta viaje de vuelta.
4. El cliente recibe el veredicto, actualiza sus contadores locales y avanza a la siguiente línea del archivo.

2.2. La Barrera del RTT (Round Trip Time)

Al analizar la gráfica de rendimiento, se observa un comportamiento sumamente característico, la línea azul que representa al modelo directo permanece completamente horizontal en la [Figura 1](#) y estancada en un rango estrecho entre las ~146 ops/sec y las ~159 ops/sec. Este estancamiento no es una anomalía de software, sino una limitación impuesta por la infraestructura física y el protocolo de comunicación. En un sistema síncrono donde un único hilo realiza peticiones secuenciales, el rendimiento máximo o tasa de operaciones por segundo (*Throughput*) está acotado estrictamente de forma inversamente proporcional al tiempo de ida y vuelta de la red o RTT (Round Trip Time) más el tiempo de cómputo en el servidor (T_{cpu}):

$$Throughput\ Max = 1 / (RTT + T_{cpu})$$

Tomando como referencia las métricas base de la red Wi-Fi/Ethernet local descritas en el entorno de pruebas, donde la latencia media de red oscila en un entorno real en torno a los 6 ms a 6.8 ms por cada intercambio completo de paquetes TCP (capa de transporte subyacente de Pyro4), la ecuación matemática arroja el siguiente límite teórico:

$$Throughput\ Teórico = 1 / 0.0065\ s \approx 153.84\ ops/sec$$

Este cálculo coincide con una precisión milimétrica con los resultados empíricos reflejados en el gráfico de barras (p. ej., **158.59 ops/sec** en el test de 1 nodo). El cliente pasa el 95% de su ciclo de vida esperando de forma bloqueante a que la red transporte los bytes, lo que inutiliza la potencia de cálculo bruta tanto del procesador Intel i7 del portátil como del Ryzen 5 de la torre.

2.3. Ineficacia de la Escalabilidad Horizontal en Entornos Síncronos

Un pilar fundamental de la computación distribuida es la escalabilidad horizontal es la capacidad de mejorar el rendimiento del sistema general añadiendo más instancias de cómputo (nodos). Sin embargo, los resultados obtenidos demuestran que en el modelo directo, añadir más nodos es completamente inútil. Al pasar de 1 nodo a 2, 4, 6 y 8 nodos lógicos distribuidos balanceados mediante la lógica Round-Robin en el cliente, el throughput no aumenta; de hecho, fluctúa ligeramente a la baja (marcando un mínimo de 146.33 ops/sec con 4 nodos) para luego volver a estabilizarse en 159.16 ops/sec con 8 nodos. La razón arquitectónica de este fenómeno es que el cuello de botella reside estrictamente en el emisor (cliente) y el medio físico (red), no en el receptor (servidor). Aunque la Torre disponga de 8 nodos Pyro4 levantados en paralelo listos para procesar compras de entradas simultáneamente, el cliente solo es capaz de enviar *una sola petición a la vez*. Cuando el cliente envía la petición al nodo1, se bloquea; cuando este responde, se desbloquea y envía la siguiente petición al nodo2, volviéndose a bloquear. Añadir hardware o replicar servidores distribuidos en el backend bajo un esquema de invocación síncrona monohilo no alivia el tiempo de tránsito de la red (RTT). Por lo tanto, el sistema se encuentra saturado por diseño arquitectónico desde el primer instante, demostrando que el acoplamiento temporal síncrono es inviable para absorber ráfagas masivas de concurrencia en entornos distribuidos.

3. Análisis del Modelo Asíncrono / Indirecto (RabbitMQ + Redis)

3.1. Filosofía de Desacoplamiento Temporal y Espacial

La segunda arquitectura evaluada rompe por completo con el paradigma del acoplamiento síncrono mediante la introducción de un modelo de comunicación indirecta guiado por eventos y mensajes. En este diseño, el cliente (`client_indirect_benchmark.py`) y el componente encargado de ejecutar la lógica de negocio (`Server_indirect.py` junto con el núcleo de `Worker.py`) no se conocen entre sí ni coinciden necesariamente en el tiempo, interactuando exclusivamente a través de un intermediario de mensajería (*Message Broker*) RabbitMQ.

El motivo del espectacular rendimiento de este modelo reside en el patrón Fire-and-Forget (Disparar y Olvidar). Cuando el script inyector de carga lee una línea del archivo de transacciones, ya no invoca un método remoto bloqueante a la espera de una validación lógica. En su lugar, empaqueta los datos de la compra en un mensaje serializado en formato JSON y lo publica instantáneamente en la cola indexada `ticket_requests` gestionada por RabbitMQ.

Una vez que RabbitMQ confirma la recepción del paquete en su búfer de memoria (un proceso que toma microsegundos), el cliente queda inmediatamente liberado para procesar la siguiente línea del benchmark. Esto elimina por completo el tiempo muerto de espera en el emisor, delegando el procesamiento real del inventario a un conjunto de procesos independientes distribuidos en el servidor de forma totalmente asíncrona.

3.2. Paralelismo Real y Escalabilidad Horizontal de los Workers

A diferencia de la línea plana del modelo directo, la línea naranja del modelo indirecto describe una curva ascendente de rendimiento en la [Figura 1](#) y la [Figura 2](#) que demuestra una escalabilidad horizontal real. Con un único worker en el servidor, el rendimiento arranca en 881.07 ops/sec, y conforme se añaden trabajadores distribuidos en la Torre, el throughput global escala de manera sobresaliente: 1203.68 ops/sec con 2 workers, 2075.10 ops/sec con 4 workers, y alcanzando picos de 2465.13 ops/sec con 6 workers.

Este incremento casi lineal es el resultado directo de la distribución eficiente de la carga de trabajo en el backend. RabbitMQ implementa un mecanismo de entrega por competencia (*Competing Consumers*), donde múltiples instancias del script `Server_indirect.py` se suscriben a la misma cola distribuidora.

Para optimizar el hardware del servidor y evitar que un worker monopolice mensajes mientras otros están inactivos, se configuró la directiva de calidad de servicio. Esta propiedad asegura que RabbitMQ distribuya los mensajes estrictamente bajo demanda de solo entregar un nuevo ticket a un trabajador cuando este ha enviado el recibo (`basic_ack`) del mensaje anterior. Al añadir más workers sobre la CPU multihilo del servidor, el sistema es capaz de consumir y procesar de forma simultánea múltiples reservas de entradas en paralelo, exprimiendo de forma real el hardware distribuido.

3.3. Resultados No Bloqueante en Redis

Redis actúa como un sumidero donde se recogen los resultados de forma eficiente. Cuando un trabajador termina su tarea, guarda el resultado usando la operación RPush, que tiene un coste constante ($O(1)$). Esto permite que el registro de los datos no penalice la velocidad de procesamiento de los workers.

Por otro lado, el cliente verifica el progreso de forma pasiva consultando el tamaño de la lista de resultados con el comando LLEN. Al no tener que esperar activamente por cada respuesta, se eliminan los bloqueos y la sobrecarga por sincronización, permitiendo que el sistema funcione a su máxima velocidad.

4. El Impacto de la Contención de Datos (Comparativa 20k vs. 60k)

El rendimiento global de un sistema distribuido está limitado principalmente por la gestión del acceso concurrente en la base de datos compartida, por encima de la velocidad de la red o la potencia del hardware. Esta limitación queda patente al comparar las pruebas de 20.000 y 60.000 transacciones, evidenciando el impacto crítico de la Contención de Datos.

4.1. Análisis del Escenario Sin Numerar (20k Tickets): Escalabilidad Limpia

En la batería de pruebas de 20.000 peticiones con formato no numerado (*unnumbered*), el sistema experimenta una progresión de rendimiento incremental muy cercana a la linealidad ideal a medida que se inyectan nuevos procesos trabajadores (Workers) en la máquina de backend:

- **1 Worker:** 881.07 ops/sec
- **2 Workers:** 1203.68 ops/sec
- **4 Workers:** 2075.10 ops/sec
- **6 Workers:** 2465.13 ops/sec
- **8 Workers:** 2691.63 ops/sec

Este rendimiento se explica por la total ausencia de conflictos entre los datos procesados. Al gestionar compras de tipo *unnumbered*, el sistema (Worker.py) simplemente reduce un contador global en Redis mediante operaciones atómicas. Al ser todos los asientos iguales, no hay disputas por recursos específicos ni bloqueos de memoria, ya que ningún proceso compite por una entrada concreta. Esta fluidez permite que Redis gestione las peticiones sin generar esperas, logrando que la arquitectura asíncrona escale de forma eficiente y mantenga un crecimiento constante en su capacidad de procesamiento hasta agotar los recursos de la red o el procesador.

4.2. Análisis del Escenario Numerado (60k Tickets): El Límite de la Saturación

El escenario cambia radicalmente al analizar la gráfica correspondiente al benchmark de 60.000 peticiones con asientos específicos numerados (*numbered*). Aunque el flujo asíncrono implementado en RabbitMQ consigue aislar al inyector de carga y mantener métricas elevadas en comparación con el modelo síncrono, la curva evolutiva de los workers revela un comportamiento asintótico y posterior degradación:

- **1 Worker:** 833.62 ops/sec
- **2 Workers:** 1194.97 ops/sec
- **4 Workers:** 2313.36 ops/sec
- **6 Workers:** 2731.25 ops/sec (Pico máximo de rendimiento)
- **8 Workers:** 2671.01 ops/sec (Rendimiento decreciente)

Al llegar a 8 instancias de procesamiento, la velocidad total (*throughput*) cae cerca de 60 operaciones por segundo, en lugar de seguir creciendo. Este retroceso ilustra los límites definidos por la Ley de Amdahl. En la prueba *numbered*, cada transacción debe verificar la

disponibilidad de un asiento específico en Redis. Al usar 8 trabajadores en paralelo, la posibilidad de que varios procesos intenten consultar, bloquear o cambiar el mismo registro de datos al mismo tiempo aumenta de forma exponencial.

5. Conclusiones Arquitectónicas Finales

Después de este análisis a fondo de los dos modelos, llegamos a las conclusiones definitivas.

5.1. Cuadro Comparativo de Rendimiento y Escalabilidad

Característica	Modelo Síncrono (Pyro4)	Modelo Asíncrono (RabbitMQ)
Acoplamiento	<i>Fuerte / Bloqueante</i>	<i>Débil / Desacoplado</i>
Limitación Principal	Latencia de Red (RTT)	Contención de Datos (Redis)
Escalabilidad Horizontal	Nula (Efecto RTT)	Alta (Hasta saturación de BD)
Throughput Máximo	~159 ops/sec	~2731 ops/sec

5.2. Superioridad del Desacoplamiento Temporal

El uso de colas de mensajes nos da una ventaja decisiva, el cliente se libera al instante, pudiendo trabajar a la máxima velocidad del procesador y memoria sin el impedimento de esperar una respuesta del servidor gracias al concepto de "disparar y olvidar".

5.3. El Rol Crítico de la Base de Datos

Tal y como hemos visto el pero caso en los sistemas asíncronos optimizados, el límite no lo impone la red, sino la base de datos. Su habilidad para gestionar sin tropezar las colisiones de datos concurrentes es lo que marca la diferencia.